

1. Overview

Insight AI - Enterprise Intelligence Studio is the as-built implementation of the **Unified Enterprise AI Agents Platform** concept: a conversational analytics surface, native to Snowflake, where a business user asks a question in plain English and receives an answer grounded in **structured warehouse data**, **unstructured documents**, an explicit measure of **data trust**, and now **demand forecasting** - with no SQL, no BI tool and no analyst in the loop. A single Cortex Agent orchestrates 13 tools and 2 MCP connectors (Jira, Gmail); a Streamlit application streams its reasoning, SQL and charts to the user in real time. The reference implementation covers insurance operations (300 policies, 250 customers, 400 claims), but the pattern is industry-neutral. This document describes the system **as built**, and marks what is still design or incomplete.

2. System Architecture

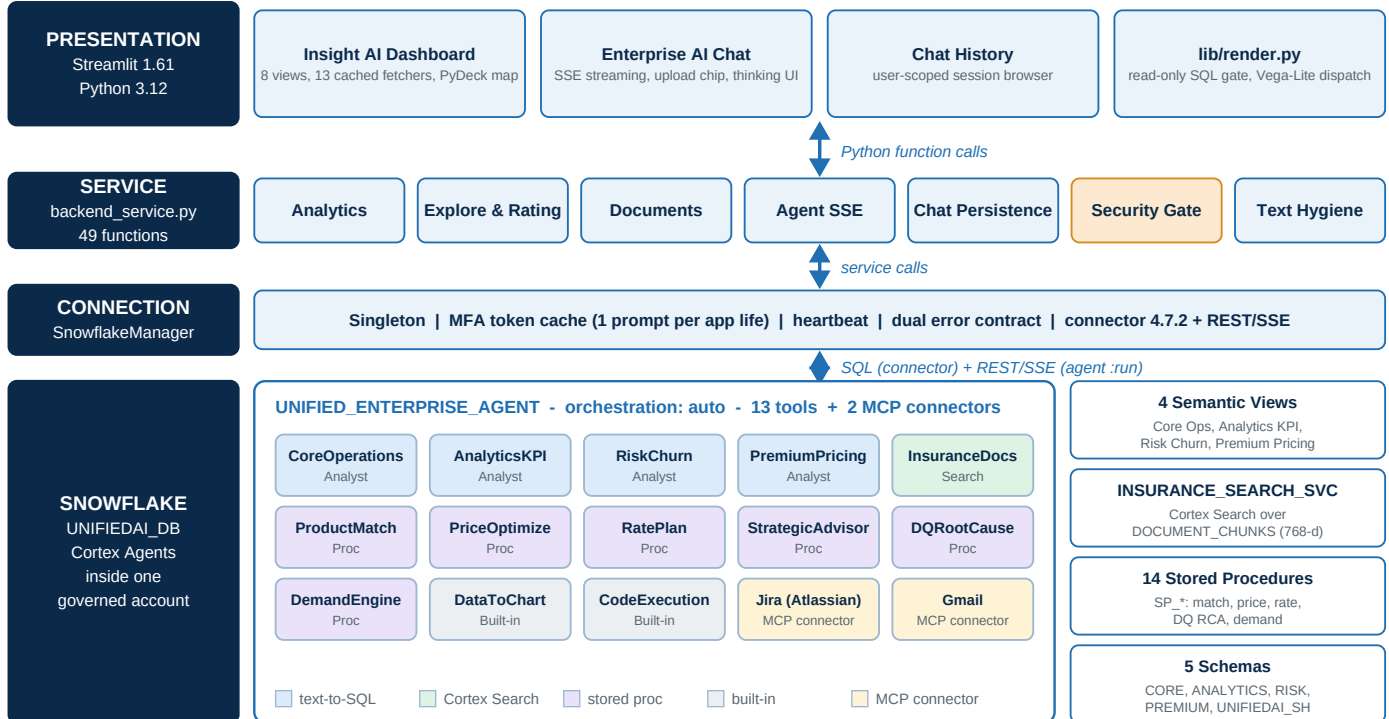


Figure 1. Four-layer as-built architecture. Every hop is a function call or a governed Snowflake API. The UI never talks to tables directly, and agent-generated SQL crosses the security gate before it can execute.

3. Design Principles

- One agent, many tools.** The agent picks its own tool and writes its own SQL or intent (*orchestration auto*). There are no question templates, so new questions need no new code.
- No number without a source.** Every displayed figure is derived from a query. Uncomputable values return None, and the UI shows nothing instead of a placeholder.
- Explainable by default.** Generated SQL is visible and can be run inline; tool activity streams live; assumptions baked into a model are labelled as assumptions, not measurements.
- Fail loudly.** Query errors surface to the user via `get_last_error()` and are never silently read as "no data".

4. Layer Responsibilities

Layer	Key components	Responsibility
Presentation	streamlit_app.py (2,767 lines), pages/1_Enterprise_AI.py, pages/2_Chat_History.py, lib/render.py, style.css	8 navigation views, chat with SSE streaming, document upload, user-scoped history, Vega-Lite and PyDeck 3-D visuals, read-only SQL gate before inline execution
Service	services/backend_service.py (2,805 lines, 49 functions, 7 groups)	All data access, agent invocation and SSE folding, document pipeline, chat persistence, security gate, markdown balancing
Connection	config/snowflake_manager.py	Singleton connection surviving Streamlit reruns; one MFA prompt per app lifetime; <code>execute_query</code> returns (None, None) on failure while <code>execute_query_checked</code> raises <code>SnowflakeQueryError</code>
Snowflake	UNIFIED_ENTERPRISE_AGENT, 4 semantic views, INSURANCE_SEARCH_SVC, 14 procedures	Reasoning, text-to-SQL, retrieval, product matching, pricing, DQ root-cause, demand forecasting; all inference runs inside the account boundary

5. Snowflake Integration

The application is a thin client over Snowflake. It reaches Snowflake through two governed paths, and every Cortex capability it uses runs inside the account.

Path	Mechanism	Detail
SQL	snowflake-connector-python 4.7.2 via the SnowflakeManager singleton	Data access, stored-procedure calls and chat persistence. One connection survives Streamlit reruns; MFA token cached (1 prompt per app life); heartbeat keeps the session alive
Agent	REST :run endpoint, stream=True, session-token auth	SSE events (text.delta, thinking.delta, status, tool_use, table, chart) are folded into one payload; no request body buffering

Snowflake capability	Used for	Resource
Cortex Agents	Orchestration of 13 tools and 2 MCP connectors (orchestration: auto)	UNIFIED_ENTERPRISE_AGENT
Cortex Analyst	Natural language to SQL over governed semantic layers	4 semantic views: Core Ops, Analytics KPI, Risk Churn, Premium Pricing
Cortex Search	Retrieval over policy wordings, contracts and uploaded documents	INSURANCE_SEARCH_SVC over DOCUMENT_CHUNKS (768-d)
CORTEX.COMPLETE	Intent classification and parameter extraction for demand questions	SP_UNIFIED_DEMAND_ENGINE
EMBED_TEXT_768	Embedding of uploaded-document chunks (batch 10, cap 400)	Document pipeline; @DOC_STAGE
SNOWFLAKE.ML.FORECAST	Multi-series demand model, one series per policy type	DEMAND_FORECAST_MODEL
Stored procedures	Product match, pricing, rate plan, recommendations, DQ root cause, demand	14 SP_* procedures in UNIFIEDAI_DB

- **Boundary:** all Cortex inference executes in the Snowflake account, and the UI calls service functions only, never tables directly.
- **To be confirmed (not stated in the as-built record):** account and region, role and grants per layer, warehouse and sizing, network policy, where the Streamlit app is deployed, and how each user's OAuth connection to Jira and Gmail is initiated from the app.

6. MCP Connectors

The agent reaches enterprise systems outside Snowflake through **MCP connectors**: Snowflake's mechanism for a Cortex Agent to discover and invoke tools on a remote Model Context Protocol (MCP) server. Two connectors are implemented, so the agent can act across systems as well as answer questions.

Connector	Provider	Use in Insight AI	Status
Jira	Atlassian	Create and track tickets from findings, e.g. a data-quality violation surfaced by DQRootCause or a StrategicAdvisor recommendation	Implemented
Gmail	Google	Send or draft email summaries of demand forecast results to stakeholders	Implemented

Connection flow: the account admin creates the MCP server on the provider side and stores its URL and OAuth credentials in a Snowflake API integration; an external MCP server object references that integration and is added to the UNIFIED_ENTERPRISE_AGENT specification; each user then connects with the provider via OAuth.

- **Governance:** MCP servers and agents are Snowflake objects governed by RBAC, and OAuth tokens are held server-side by Snowflake.
- **Boundary:** the built tools keep all inference inside the account; MCP calls exchange data with external providers, so that guarantee covers inference, not connector traffic. The read-only SQL gate covers SQL only, so ticket and email actions are outside it.

7. Agent Tools

Tool	Type	Backing resource	Purpose
CoreOperations, AnalyticsKPI, RiskChurn, PremiumPricing	Cortex Analyst	4 semantic views	Policies, claims, KPIs, churn/risk and premium questions to SQL
InsuranceDocs	Cortex Search	INSURANCE_SEARCH_SVC	Policy wordings, contracts and uploaded documents
ProductMatch, PriceOptimize, RatePlan	Stored proc	SP_PRODUCT_MATCH_AND_SAVE, SP_PRICE_OPTIMIZE, SP_RATE_PLAN	Ranked plan matches with reasoning, competitive price position, ratings
StrategicAdvisor, DQRootCause	Stored proc	SP_GENERATE_RECOMMENDATIONS, SP_DQ_ROOT_CAUSE_ANALYSIS	Recommendations, conversational data-quality root-cause analysis
DemandEngine	Proc + ML.FORECAST	SP_UNIFIED_DEMAND_ENGINE + DEMAND_FORECAST_MODEL	Single NL front end routing to train / forecast / show-results (8D)
DataToChart, CodeExecution	Built-in	None	Vega-Lite chart specs; sandboxed computation
Jira, Gmail	MCP connector	External MCP servers (API integration + OAuth)	Ticket and email actions from agent findings (6)

8. Key Workflows

A. Streaming conversation. The app calls the agent :run REST endpoint with stream=True; a generator over iter_lines() yields SSE events without buffering the body, folding text.delta, thinking.delta, status, tool_use, table and chart events into one payload while a callback drives the live UI.

B. Document intelligence. The agent REST API has no attachment channel, so retrieval runs app-side:

Upload	Sanitise filename	PUT to @DOC_STAGE	pypdf extract	Chunk ~1000 chars, 150 overlap	EMBED_TEXT_768 batch 10, cap 400	Store chunks, refresh index	Filtered SEARCH_PREVIEW, passages injected in prompt
--------	-------------------	-------------------	---------------	--------------------------------	----------------------------------	-----------------------------	--

C. Data trust. Because the DQ rule tables are empty, quality is measured directly on CORE: 13 field checks and 9 validity rules yield 95.1% completeness and 93.9% validity, with 58 violations (53 approved-above-claimed, 5 resolved-before-claimed), each naming its table, rule and failing row count.

D. Demand forecasting.

- **One procedure, whole engine:** SP_UNIFIED_DEMAND_ENGINE handles everything; there is no separate train or forecast API.
- **Intent step:** CORTEX.COMPLETE classifies the question into an intent (TRAIN, FORECAST, SHOW_RESULTS or GENERAL) and extracts up to seven parameters; the agent tool passes the question through unparsed so this step works.
- **Train:** rebuilds a multi-series SNOWFLAKE.ML.FORECAST model, one series per policy type, over 9 months of history.
- **Forecast:** projects up to 12 months, clamping every value to zero or above. It is reachable only conversationally today; no dashboard panel yet reads the result tables.

9. Security and Governance

Control	Implementation	Threat closed
Read-only SQL gate	assert_read_only_sql() blocks 19 keywords after literals and comments are stripped; REPLACE and GET intentionally allowed (real functions)	LLM-generated DML/DDDL running on click
Bound parameters	All filters and all chat persistence; only int-coerced row limits are interpolated	Injection via search box, filters, chat content
Filename sanitisation	Strips drive letters and backslash paths, whitelists [A-Za-z0-9._-], caps 180 chars	SQL / stage injection via upload name
Per-user history	USER_NAME predicate on list, fetch and delete	Cross-user disclosure; delete by guessed session ID
Boundary	All Cortex inference executes in Snowflake; session-token auth to agent API	Data leaving the governance perimeter
MCP connectors	RBAC on MCP server objects and agents; OAuth tokens stored server-side by Snowflake	Unauthorised use of connector tools; credential exposure